SIMATS ENGINEERING

CO2_AT3 Error Analysis Task

COURSE NAME: Software Testing **COURSE CODE: CSA37**

COURSE OUTCOME COVERED

CO2: *Design effective test cases using black-box, white-box testing techniques and to control methods flow.*

Assessment Tool Weightage: 20%

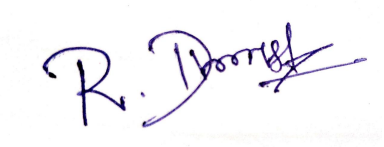
Sl. No	Questions	Bloom's Level
1	A tester designs test cases for an online shopping system using equivalence partitioning but incorrectly groups input ranges (e.g., valid and invalid inputs mixed in one class). Additionally, boundary values are not considered. Analyze the given test design and identify all errors in equivalence class partitioning and boundary value analysis. Examine the root causes of these mistakes with proper reasoning. Propose corrected test cases with proper classification and boundary coverage. Justify how your corrections improve defect detection effectiveness.	3
2	A QA engineer performs random testing on a banking application and claims high coverage without using any structured techniques. Critical defects in transaction limits and boundary conditions are missed. Evaluate the testing approach and identify errors in strategy and coverage assumptions. Analyze the root causes behind missing critical defects. Suggest appropriate black box testing techniques to correct the approach. Justify how these techniques ensure better test adequacy and reliability.	3
3	A developer performs white box testing but creates an incomplete control flow graph, missing decision nodes and loops, resulting in poor path coverage. Analyze the errors in the control flow graph and path testing approach. Identify the root causes for inadequate test coverage. Propose corrections by constructing a proper control flow representation and identifying all independent paths. Justify how improved path testing ensures better coverage and defect detection.	5

4	A testing team reports 100% statement coverage for a module but fails to detect logical errors in conditional statements. Evaluate the adequacy of the testing performed and identify gaps in coverage criteria. Analyze why statement coverage alone was insufficient. Recommend improved white box testing strategies such as branch or path coverage. Justify how the revised approach enhances overall test effectiveness and error detection.	5
5	A tester uses Equivalence Class Partitioning for an age-validation field (0-100) but defines only two classes: Negative numbers and Positive numbers. Analyze the errors in this partitioning strategy. Identify why this fails to account for valid vs. invalid age limits and the absence of Boundary Value Analysis. Propose a corrected set of partitions and justify how they ensure more robust input validation.	3
6	A developer claims full Condition Coverage for a complex `if(A && B	
7	A QA team applies Decision Table Testing for a discount system but misses several combinations of "Premium Member" and "Coupon Applied," leading to price calculation errors. Analyze the errors in the construction of the decision table. Identify why the current approach is incomplete. Propose a revised, exhaustive decision table and justify its effectiveness in preventing production defects.	3
8	During unit testing of a looping structure, a tester checks only the first and last iterations (n=1 and n=max). Defects occurring at n=0 and n=max+1 are missed. Evaluate the errors in this Loop Testing strategy. Analyze the root causes behind these omissions. Recommend a corrected approach for testing simple and nested loops, and justify how it ensures path integrity.	5
9	An engineer utilizes Error Guessing as the primary testing technique for a safety-critical braking system, ignoring structured black box methods. Identify the errors in relying solely on ad-hoc techniques for high-risk modules. Analyze the impact on test adequacy. Suggest a combination of BVA and State Transition Testing to fix the strategy, justifying the increased safety assurance.	5

10	A tester designs a Control Flow Graph (CFG) for a module but fails to represent the "else" branches of multiple nested if statements, leading to 100% statement coverage but 0% branch coverage on error paths. Analyze the errors in graph construction. Identify the root causes for the missing nodes. Propose a corrected CFG and independent paths to ensure branch adequacy.	5

Code of Conduct:

*I **Dhaanush Rajan(192421162)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

ANSWER

Question 1: Equivalence Partitioning and Boundary Value Analysis in Online Shopping

Errors in the test design

- Valid and invalid values have been mixed into the same equivalence class. An equivalence class should contain inputs expected to be handled in the same way by the system.
- Boundary values have not been tested. Defects often occur at the minimum and maximum permitted values or immediately outside them.
- The input field, its permitted range, and the expected response for each class have not been specified. Without these, the classes cannot be checked for completeness or mutual exclusivity.

Root causes

The tester may have treated equivalence partitioning as simply dividing a range into groups without defining expected system behaviour. The tester may also have assumed that a representative value from the middle of a range is sufficient. That assumption misses off-by-one errors, incorrect comparison operators, and range-validation defects.

Corrected example and test cases

Assume an online shop accepts a quantity from 1 through 10 items per order. This is an illustrative rule because the assessment does not specify the shop's actual limits.

Equivalence classes

- Invalid class 1: quantity less than 1 (for example, −1 or 0). Expected: reject with a validation message.
- Valid class: quantity from 1 through 10 inclusive (for example, 5). Expected: accept and calculate the order total correctly.
- Invalid class 2: quantity greater than 10 (for example, 11). Expected: reject or request a permitted quantity.

Boundary-value test cases

- 0: immediately below the minimum; expected rejection.
- 1: minimum valid value; expected acceptance.
- 2: just above the minimum; expected acceptance.
- 9: just below the maximum; expected acceptance.
- 10: maximum valid value; expected acceptance.
- 11: immediately above the maximum; expected rejection.

Justification

Equivalence partitioning reduces redundant tests by selecting representative values from behaviourally similar classes. Boundary value analysis targets the edges where errors are especially likely. Together they provide more systematic coverage than a mixed, unstructured set of values and improve the chance of detecting invalid acceptance, valid rejection, and off-by-one defects.

Question 2: Evaluating Random Testing for a Banking Application

Errors in the strategy

- Randomly selected tests do not guarantee that important transaction limits or boundary values will be exercised.
- Claiming high coverage without measurement confuses the number of tests executed with coverage of requirements, input classes, decisions, or risk areas.
- The strategy does not document test objectives, expected results, reproducible test data, or coverage evidence.
- Critical financial scenarios may be underrepresented because random selection gives no assurance that rare but important combinations will occur.

Root causes of missed defects

Transaction limits often fail at exact thresholds or immediately above and below them. Random values may cluster in ordinary ranges. In addition, without requirements-based test design, the team may not know which boundaries, invalid states, and combinations must be tested.

Recommended black-box techniques

- Equivalence Class Partitioning: divide transaction amounts into valid and invalid groups, including negative amounts, zero, permitted amounts, and amounts beyond the limit.
- Boundary Value Analysis: if the allowed transfer amount is illustratively ₹100–₹50,000, test ₹99, ₹100, ₹101, ₹49,999, ₹50,000, and ₹50,001. Confirm the actual limits from the requirements before execution.
- Decision Table Testing: cover combinations such as sufficient/insufficient balance, valid/blocked beneficiary, within/above transfer limit, and valid/expired authentication.
- State Transition Testing: test account and transaction states such as initiated, pending, successful, failed, reversed, and timed out. Check that illegal transitions are rejected.
- Use-case or scenario testing: exercise a complete transfer from authentication through confirmation, ledger update, notification, and failure recovery.

Justification

Structured techniques make test selection traceable to requirements and risk. Random testing can still supplement them to discover unexpected combinations, but it should not replace planned boundary, decision, and state tests. Coverage should be reported with a defined measure and evidence.

Question 3: Correcting an Incomplete Control Flow Graph and Path Testing

Errors and root causes

- Decision nodes are omitted, so true and false outcomes are not represented separately.
- Loops are omitted or represented as straight-line code, hiding repeated execution and loop-exit behaviour.
- Edges may skip statements or branches, making the graph inconsistent with the program's control flow.
- Independent paths have not been identified, so path coverage is incomplete and may be overstated.

Illustrative corrected control-flow representation

Because the assessment does not provide source code, use this example: Start → read x → decision (x > 0?) → true: set result = "positive"; false: decision (x == 0?) → true: set result = "zero"; false: set result = "negative"; all three result assignments join at Output → End. For a loop example, add decision (i < n?) with a true edge into the loop body and a back edge from the body to the decision; the false edge exits the loop.

Independent paths for the illustrative example

- Path 1: Start → read x → x > 0 is true → positive assignment → Output → End.
- Path 2: Start → read x → x > 0 is false → x == 0 is true → zero assignment → Output → End.
- Path 3: Start → read x → x > 0 is false → x == 0 is false → negative assignment → Output → End.

How to test

Choose x = 5 for Path 1, x = 0 for Path 2, and x = −3 for Path 3. For a loop, also test zero iterations, one iteration, and multiple iterations where the specification permits them. Compare actual results with expected results.

Justification

A complete CFG exposes every decision outcome, loop-back edge, and exit. Independent-path testing aims to execute each linearly independent path at least once. It improves branch and decision coverage and helps reveal missing logic, incorrect conditions, and loop termination defects. Exhaustive path coverage may be impossible when loops or many decisions create an enormous or infinite path set, so use basis paths and risk-based additional tests.

Question 4: Why 100% Statement Coverage Is Not Sufficient

Evaluation and gaps

Statement coverage confirms that every executable statement ran at least once. It does not confirm that every decision outcome was taken. A test can execute a statement inside an if block while never testing the alternative outcome. Therefore, 100% statement coverage does not prove that conditional logic is correct.

Example

Consider: if (balance >= amount) { approveTransfer(); } else { rejectTransfer(); }. A test with balance = ₹1,000 and amount = ₹100 executes approveTransfer(), but it does not test the else branch. A defect in the insufficient-balance handling could remain undetected.

Recommended improvements

- Branch/decision coverage: test both true and false outcomes of every decision. For the example, test balance ₹1,000 with amount ₹100 and balance ₹50 with amount ₹100.
- Condition coverage: make each Boolean subcondition evaluate to both true and false at least once. This is especially important for compound expressions.
- Modified Condition/Decision Coverage (MC/DC), where appropriate: demonstrate that each condition can independently affect the overall decision outcome. This is useful in high-assurance systems.
- Path coverage: test important combinations of branches, while recognising that complete path coverage

can be impractical for complex programs.

Justification

Branch coverage directly addresses the missing alternative outcomes that statement coverage can overlook. Condition and path-oriented techniques add confidence for compound logic and interactions. The team should report the selected criterion and its measured coverage rather than treating one metric as proof of correctness.

Question 5: Correct Equivalence Classes for an Age Field (0–100)

Errors in the original partition

- "Negative numbers" and "positive numbers" do not distinguish valid ages from invalid positive values above 100.
- Zero is often treated separately in informal reasoning, but it must be explicitly classified according to the stated rule. Here, if the accepted range is 0–100 inclusive, zero is valid.
- The classes do not specify the maximum limit or values just outside the range.
- No boundary value analysis is included.
- **Corrected equivalence classes**
- Invalid class 1: age < 0, such as −1; expected rejection.
- Valid class: 0 ≤ age ≤ 100, such as 35; expected acceptance.
- Invalid class 2: age > 100, such as 101; expected rejection.
- Invalid data-type class: non-numeric text such as "abc", if the field requires a numeric age.
- Additional format classes should be added if the requirement defines whether decimals, blanks, or null values are allowed.

Boundary-value tests

- −1: just below minimum, reject.
- 0: minimum allowed age, accept.
- 1: just above minimum, accept.
- 99: just below maximum, accept.
- 100: maximum allowed age, accept.
- 101: just above maximum, reject.

Justification

The corrected classes separate inputs by expected behaviour and cover both sides of the permitted interval. Boundary tests detect incorrect comparisons such as age > 0 instead of age >= 0, or age < 100 instead of age <= 100.

Question 6: Condition Coverage for a Compound Boolean Expression

Source limitation

The uploaded question is incomplete: the visible text ends at "A developer claims full Condition Coverage for a complex if(A && B". No further scenario or full expression is visible. The following is an illustrative

explanation of the stated condition-coverage issue, not a reconstruction of missing source text.

Meaning and common error

For the expression if (A && B), condition coverage requires each atomic condition, A and B, to evaluate to both true and false across the test set. Merely making the overall decision true and false is not enough to prove condition coverage.

Illustrative test set

- Test 1: A = true, B = true → A && B is true.
- Test 2: A = false, B = true → A && B is false; A has now been false while B has been true.
- Test 3: A = true, B = false → A && B is false; B has now been false while A has been true.

Coverage reasoning and caution

Across these tests, both A and B take true and false values. In programming languages with short-circuit evaluation, B may not actually be evaluated when A is false. If the test objective requires each condition to be evaluated, choose tests and instrumentation carefully, and verify executed condition outcomes. The precise expected answer may need adjustment if the original question contains additional conditions or a different expression.

Justification

Condition coverage is stronger than statement coverage for Boolean subexpressions because it requires each condition to take both outcomes. For high-risk compound logic, decision coverage and MC/DC may also be appropriate. The complete original wording is needed to confirm any additional requested analysis.

Question 7: Decision Table Testing for a Discount System

Errors in the existing approach

- Some combinations of Premium Member and Coupon Applied have been omitted.
- Actions and expected discounts may not be specified for every condition combination.
- Rules may overlap or leave gaps, creating inconsistent or undefined price calculations.
- The team may have tested individual conditions but not their combinations.

Revised exhaustive decision table

Assume, for illustration, that a premium member receives a 10% discount and a valid coupon provides an additional 5% discount. Assume the discounts are additive and calculated on the original price; the actual business rules must be confirmed. “Coupon Applied = Yes” here means a valid coupon is present.

Condition / Action	Rule 1	Rule 2	Rule 3	Rule 4
Premium member?	No	No	Yes	Yes
Valid coupon applied?	No	Yes	No	Yes
Member discount	0%	0%	10%	10%

Coupon discount	0%	5%	0%	5%
Total illustrative discount	0%	5%	10%	15%
Expected action	Charge full price	Apply coupon discount	Apply member discount	Apply both discounts

Example: for an original price of ₹1,000, the four rules yield ₹1,000, ₹950, ₹900, and ₹850 respectively under the illustrative additive rule.

Justification

Two Boolean conditions create $2^2 = 4$ combinations. Testing all four rules makes omissions visible and provides a direct check of the calculation for each combination. If coupons can be invalid, expired, product-specific, or non-stackable, add the relevant condition(s) and rules rather than assuming the four-rule table is sufficient for the real system.

Question 8: Correct Loop Testing Strategy

Errors and root causes

- Testing only n = 1 and n = max omits zero iterations and the just-over-limit case.
- The tests do not verify loop-entry and loop-exit conditions adequately.
- Nested loops introduce interactions between outer and inner loop counts; testing only their extremes can miss defects in initialization, updates, and termination.
- The test plan does not define what should happen when the iteration count is invalid or exceeds the supported limit.

Recommended simple-loop tests

- n = 0: the body should execute zero times if zero is permitted; verify correct initial/final state.
- n = 1: minimum positive iteration count.
- n = 2: a small typical count to expose update errors.
- n = max − 1: just below the supported maximum.
- n = max: maximum permitted count.
- n = max + 1: verify rejection, safe handling, or specified behaviour beyond the limit.
- Invalid values such as negative n should be tested if they can reach the loop through input or internal calls.

Nested-loop approach

- Begin with the outer loop at its minimum and keep the inner loop at a typical value; then vary the inner loop.
- Test the inner loop at 0, 1, typical, maximum, and maximum + 1 where meaningful.
- Test the outer loop at 0, 1, typical, and maximum while keeping the inner loop simple.
- Use a limited set of combinations to isolate each loop, then test selected interactions and termination conditions.

- Check counters, array indices, accumulators, and state changes after each loop.

Justification

The corrected strategy exercises loop entry, body execution, normal termination, and invalid limits. For nested loops, varying one loop at a time helps localise defects and reduces the combinatorial test burden. Expected behaviour at n = max + 1 must come from the specification; it should not be assumed to be acceptance.

Question 9: Error Guessing in a Safety-Critical Braking System

Problems with relying only on error guessing

- Error guessing depends heavily on the tester's experience and can omit systematic, requirement-derived cases.
- It provides weak evidence of completeness and is difficult to measure or reproduce.
- Safety-critical braking behaviour includes timing, sensor inputs, operating modes, fault handling, and state transitions that should be explicitly covered.
- An undetected defect could cause failure to brake, unintended braking, delayed braking, or unsafe transitions.

Use Boundary Value Analysis

Identify specification-defined thresholds such as minimum and maximum sensor readings, speed limits, pressure thresholds, response-time limits, and timing windows. For each threshold T, test values immediately below T, at T, and immediately above T, where those values are meaningful and safe to simulate. Do not invent real braking-system thresholds; use approved requirements and bench/simulation environments.

Use State Transition Testing

- Define states such as Standby, Braking Requested, Braking Active, Fault Detected, Degraded Mode, and Recovery, subject to the actual system specification.
- Test valid transitions, such as Braking Requested → Braking Active.
- Test invalid or prohibited transitions, such as Recovery without the required safety checks.
- Inject faults such as a sensor signal loss or implausible reading in a controlled simulation and verify the specified safe response.
- Check timing and event ordering, including simultaneous requests or a fault during braking.

Combined strategy and justification

Use requirements-based tests and risk analysis as the foundation, apply BVA to numeric thresholds and timing limits, apply state-transition tests to operating modes and fault recovery, and retain error guessing as an additional exploratory technique. Independent review, traceability, controlled test environments, and documented evidence are essential. This layered approach provides stronger safety assurance than ad-hoc testing alone.

Question 10: Correcting a CFG with Missing Else Branches

Errors in graph construction

- Missing else edges mean the CFG does not represent all possible decision outcomes.
- Nested decisions may be drawn as though the true branch were the only possible route.
- Tests that execute all visible statements can still miss error-handling statements and false outcomes.
- Reporting 100% statement coverage while branch coverage on error paths is 0% reveals a coverage-criterion gap, not a contradiction.

Illustrative corrected CFG

Since no source code is supplied, consider this example: Start → decision D1: input valid? If false, go to Error 1 → End. If true, proceed to decision D2: funds sufficient? If true, execute Transfer → Success → End. If false, execute Error 2 → End. Both outcomes of D1 and D2 must appear as distinct edges. If the actual code contains further nested decisions, each requires its own true and false edges.

Independent paths for the illustrative CFG

- Path 1: Start → D1 true → D2 true → Transfer → Success → End.
- Path 2: Start → D1 true → D2 false → Error 2 → End.
- Path 3: Start → D1 false → Error 1 → End.

Test data

- Path 1: provide a valid input and sufficient funds.
- Path 2: provide a valid input but insufficient funds.
- Path 3: provide invalid input, such as a missing or malformed required field, according to the specification.

Justification

Representing every branch makes error handling visible and allows branch adequacy to be measured. The test suite should execute each true and false outcome of each decision at least once, then add path tests for important interactions. These paths are illustrative; the actual CFG and path list must be derived from the module's source code.